

第9讲 存储器管理(2/3)

清华大学电子工程系
马洪兵

OPERATING SYSTEM

大纲

01

非连续分配存储管理

02

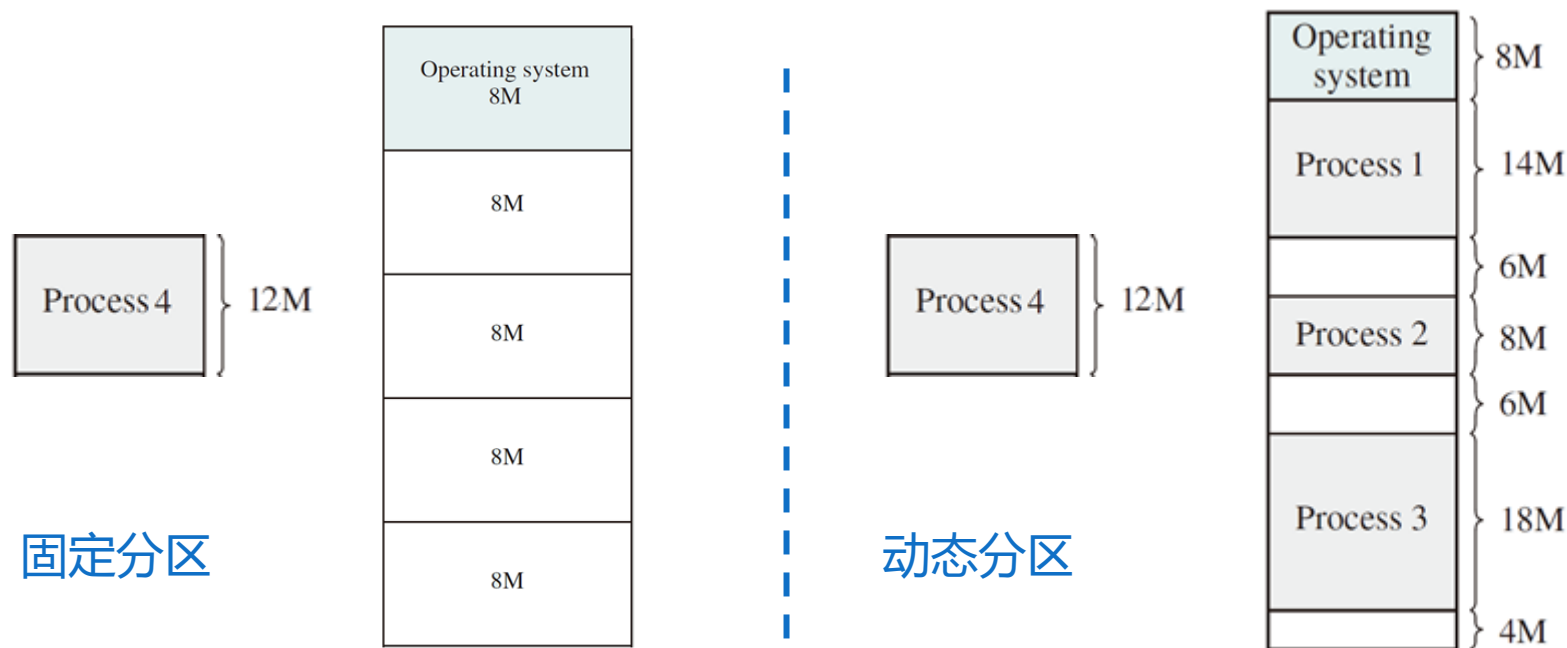
简单页式存储管理

03

虚拟页式存储管理

连续存储管理的局限

- 采用连续存储管理技术时，进程在内存中必须连续存放，使进程受到可用连续内存容量的限制



非连续分配存储管理

- 突破进程在内存中必须连续存放的限制，导致非连续分配（离散分配）存储管理技术的诞生
- 非连续分配：运行时将一个进程的程序和数据离散存储在多个独立的分配单位中
- 非连续分配技术包括：
 - 页式存储管理技术
 - 段式存储管理技术
 - 段页式存储管理技术

页式存储管理

- 支持页式存储管理的处理器将物理内存划分为固定大小的块，称为页（page），操作系统据此将进程的逻辑地址空间划分为同样大小的页
- 以页为单位实现内存的分配和回收，通过页表实现逻辑地址到物理地址的转换
- 页式存储管理技术可以按照是否支持虚拟内存分为简单页式存储管理和虚拟页式存储管理。简单页式存储管理应用场景主要是嵌入式系统，现代通用操作系统一般采用虚拟页式存储管理

段式存储管理

- 将程序和数据划分为逻辑上独立的段（Segment），每个段代表一个功能单元，如代码段、数据段、堆栈段等。不同的段在物理内存中不需要连续
- 属于过时的存储管理技术，目前主流的通用处理器大多采用分页式存储管理，纯段式存储管理在现代系统中已较少使用
- 部分嵌入式或专用处理器提供对段式管理的硬件支持
 - 某些版本ARM通过分段实现内存保护机制
 - RISC-V可通过自定义扩展实现分段，但标准架构依赖分页

段页式存储管理

- 段页式（Segmentation with Paging）内存管理是一种结合分段和分页两种技术的混合内存管理方案
- 分段：按程序逻辑（如代码、数据、堆栈等）划分内存，每段独立管理
- 分页：将每个段进一步划分为固定大小的页，物理内存按页分配
- 现代处理器中，采用段页式存储管理的处理器架构主要是x86，其他主流架构，包括ARM、RISC-V、MIPS，通常都采用纯分页管理

段页式存储管理

- x86架构从16位时代引入分段，32位时代加入分页机制，形成了段页式管理。出于兼容性考虑，现代x86处理器现代仍然提供段页式存储管理硬件支持，但是现代操作系统实际上都绕过了分段而采用分页管理
- 32位模式下分段不可绕过，但操作系统通常将CS、DS、ES、SS等设为平坦模式（Flat Model），基址为0，界限为最大值，等效于绕过分段
- 64位模式下分段的大部分功能被弱化，CS、DS、ES、SS的基址固定为0，仅保留必要的段寄存器如FS、GS用于线程本地存储TLS

大纲

01

非连续分配存储管理

02

简单页式存储管理

03

虚拟页式存储管理

简单页式存储管理

- 简单页式 (simple paging) 管理是基于实存 (real memory) 的页式存储管理。当进程加载时, 以页为单位进行分配, 并按进程的页数多少来分配
- 逻辑上相邻的页, 物理上不一定相邻

简单页式存储管理

- 逻辑页号和物理页号都是从0开始编址，页内地址相对于0编址
- 页的划分是由系统自动完成的，对用户透明
- 为区分程序逻辑地址空间的页，物理内存的页通常称为页框(page frame)

简单页式存储管理

- 指令所给出逻辑地址分为两部分：逻辑页号，页内偏移地址

逻辑页号	页内偏移地址
------	--------

逻辑地址格式

- 物理地址同样分为两部分：物理页框号，页内偏移地址

物理页框号	页内偏移地址
-------	--------

物理地址格式

简单页式存储管理

■ 逻辑页号P和页内地址d的计算公式

$$P = \lfloor A/L \rfloor$$

$$d = A \bmod L$$

A ——逻辑地址, L ——页面大小(为2的幂次)

■ 例如, 页面大小为1KB, 地址 $A=2170$, 则求得 $P=2$, $d=122$

$$2170 = \begin{array}{|c|c|c|c|c|c|c|c|c|c|} \hline 1 & 0 & 0 & 0 & 0 & 1 & 1 & 1 & 1 & 0 & 1 & 0 \\ \hline \end{array}$$

2 122

简单页式存储管理

0	A.0
1	A.1
2	A.2
3	A.3
4	B.0
5	B.1
6	B.2
7	C.0
8	C.1
9	C.2
10	C.3
11	
12	
13	
14	

0	A.0
1	A.1
2	A.2
3	A.3
4	
5	
6	
7	C.0
8	C.1
9	C.2
10	C.3
11	
12	
13	
14	

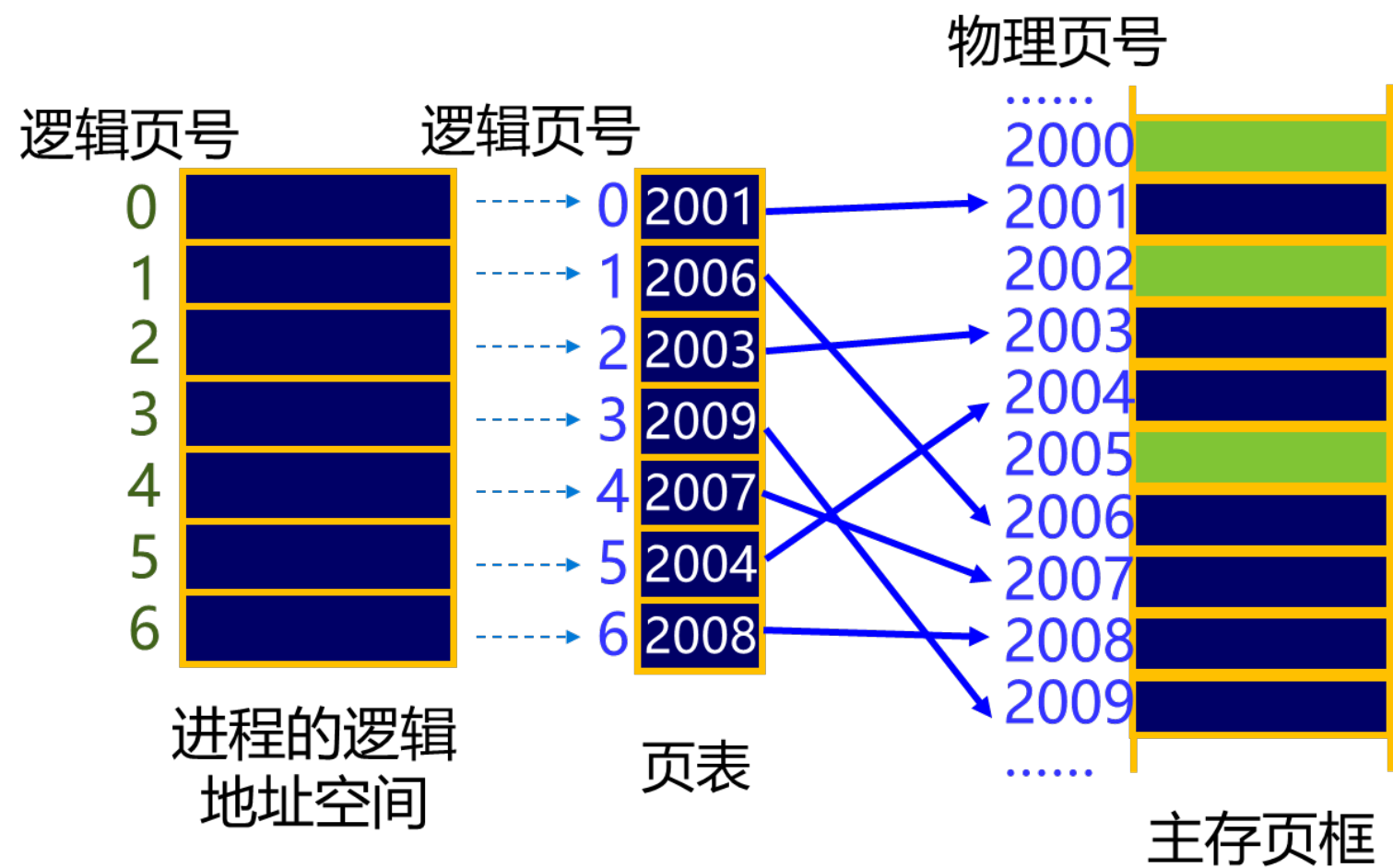
0	A.0
1	A.1
2	A.2
3	A.3
4	D.0
5	D.1
6	D.2
7	C.0
8	C.1
9	C.2
10	C.3
11	D.3
12	D.4
13	
14	

简单页式存储管理

- **进程页表**：每个进程有一个页表，在逻辑页与物理页框之间建立起对应关系
 - 逻辑页号(本进程的地址空间)——>物理页框号(实际内存空间)
- **物理页框表**：整个系统有一个物理页框表，描述物理内存空间的分配使用状况
 - 数据结构：位图，空闲页面链表
- **请求表**：整个系统有一个请求表，描述系统内各个进程页表的位置和大小，用于地址转换，也可以结合到各进程的PCB里

简单页式存储管理

■ 页表



地址变换

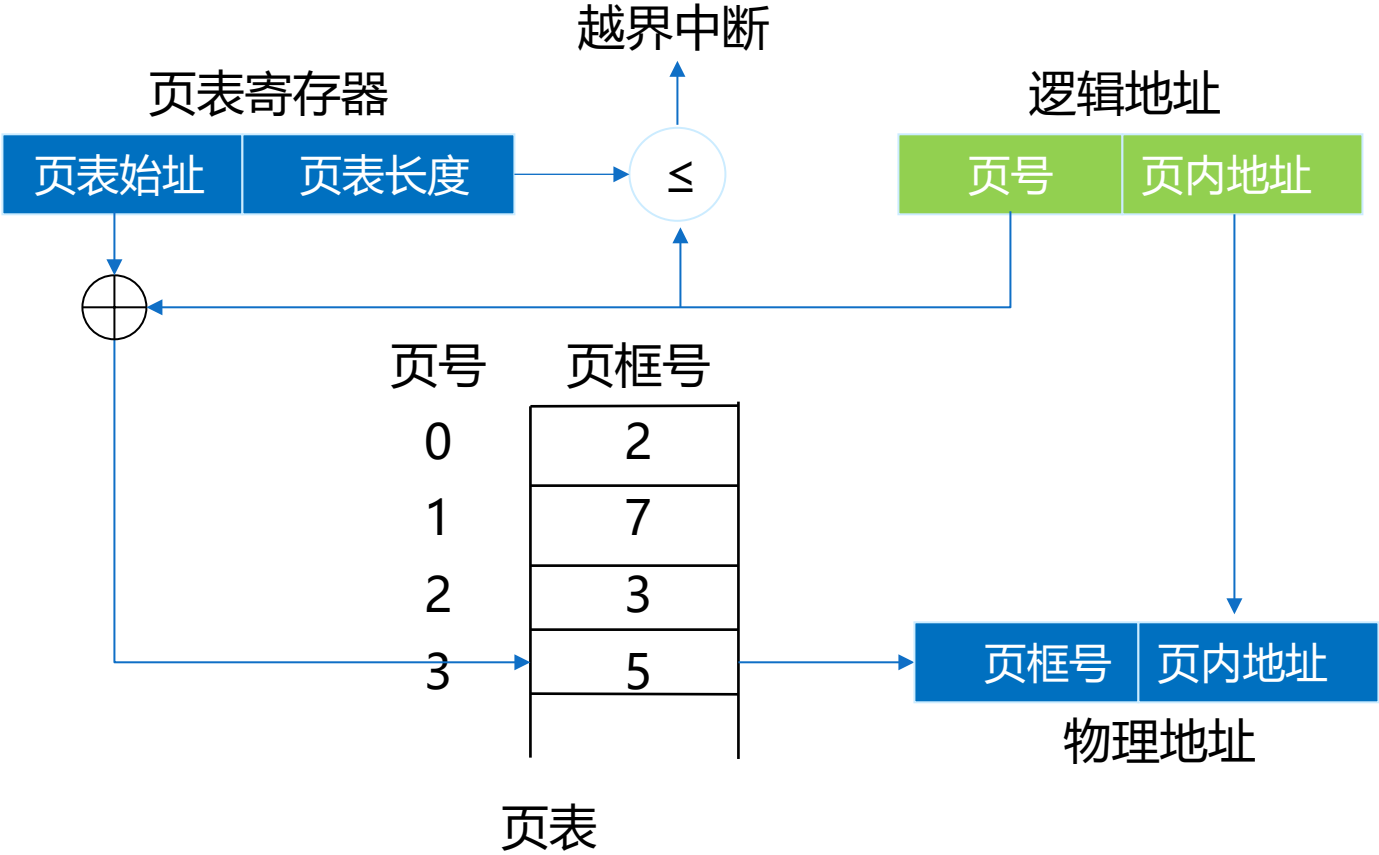
- 地址变换(地址映射)即从逻辑地址到物理地址的变换过程
- 进程的页表存放在内存系统区的一个连续空间中
- 处理器的页表寄存器(Page-Table Register, PTR)中存有页表在内存的首地址和页表长度(即页数)

地址变换

- 处理器的地址变换机构自动将逻辑地址分为页号和页内地址
- 根据页号查询页表——页表首地址 + 页号 × 页表项长度
- 找到该页对应的物理页框号，计算得到物理地址

$$\text{物理地址} = \text{物理页框号} \times \text{页面大小} + \text{页内地址}$$

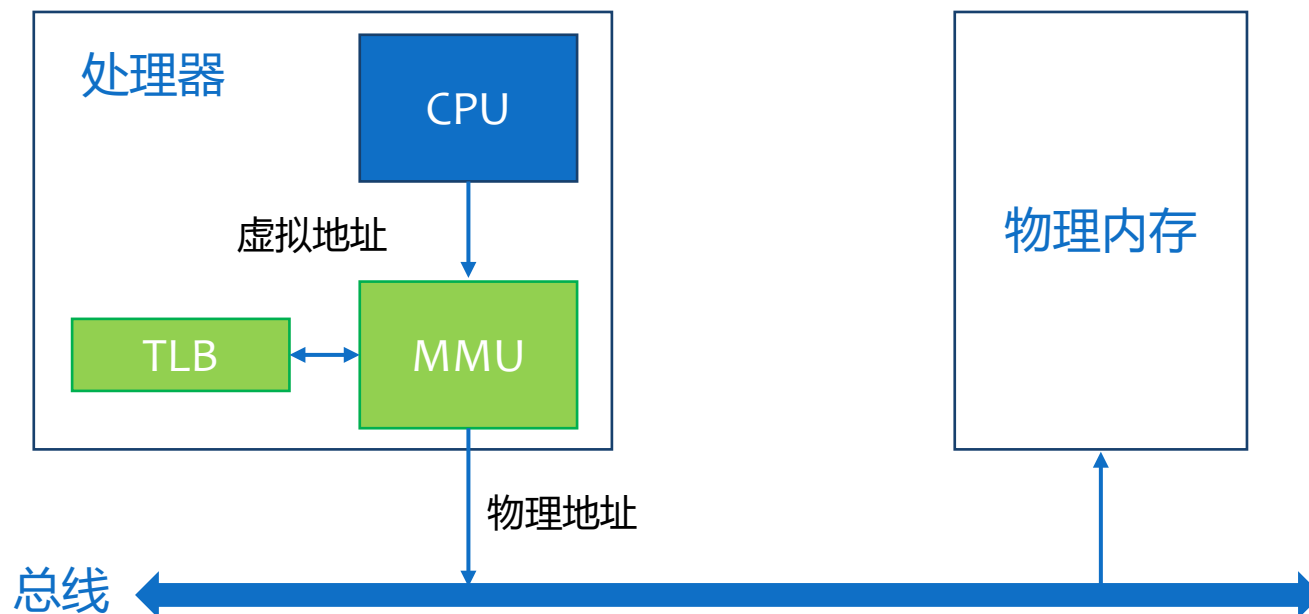
地址变换



页式管理的硬件支持

■ MMU(Memory Management Unit, 内存管理单元)

- 管理硬件页表寄存器
- 分解逻辑地址
- 访问页表
- 管理快表TLB



地址变换

- 例：已知某分页系统，主存容量为64k，页面大小为1k，对一个4页大的进程，第0、1、2、3页被分配到内存的2、4、6、7号页框中。将十进制的逻辑地址1023、2500、4500转换成物理地址
1. $1023 \div 1024 = 0 \dots\dots 1023$ ，页号为0，页内地址为1023。对应的物理页框号为2，故物理地址为 $2 * 1k + 1023 = 3071$
 2. $2500 \div 1024 = 2 \dots\dots 452$ ，页号为2，页内地址为452。对应的物理页框号为6，故物理地址为 $6 * 1k + 452 = 6596$
 3. $4500 \div 1024 = 4 \dots\dots 404$ ，页号为4，页内地址为404。因为页号不小于页表长度，故产生越界中断

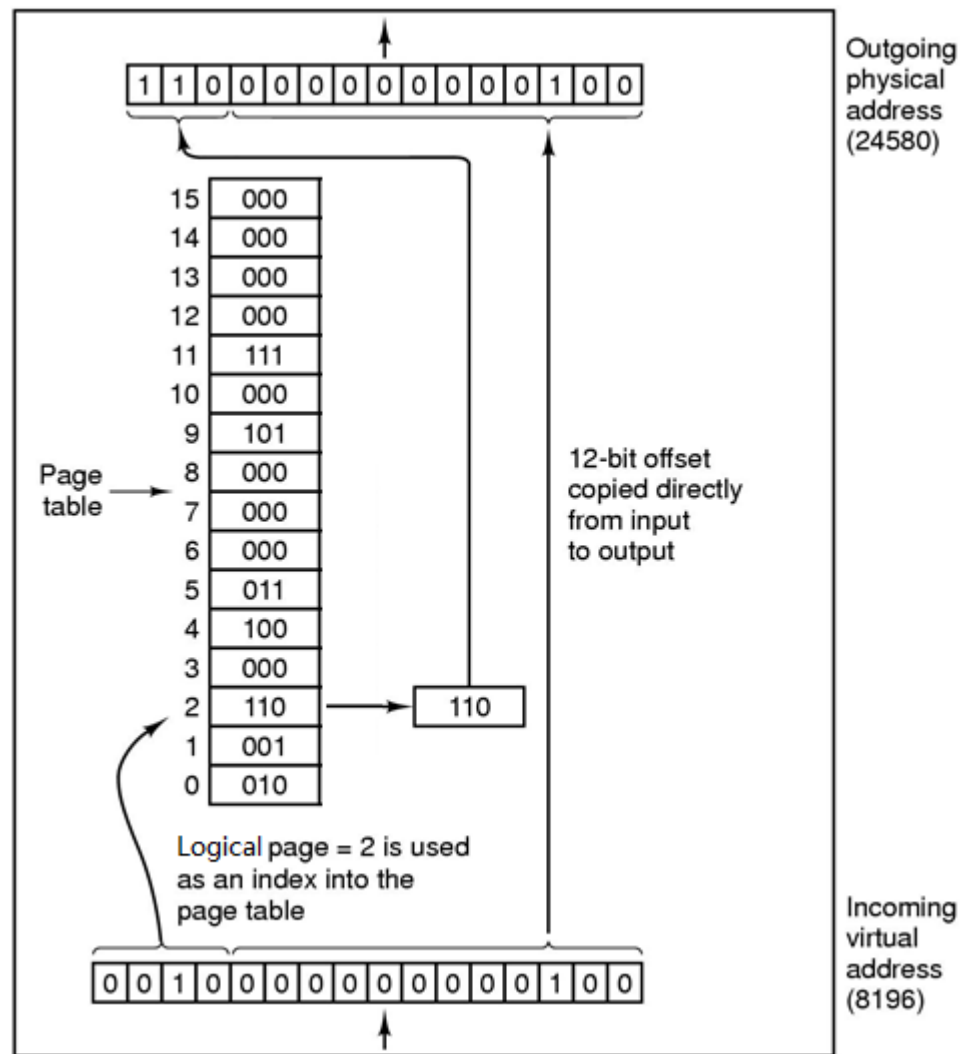
地址变换

■ MMU的内部操作

■ 例

■ MOVE REG, 8196

8196 = 010 0000 0000 0100

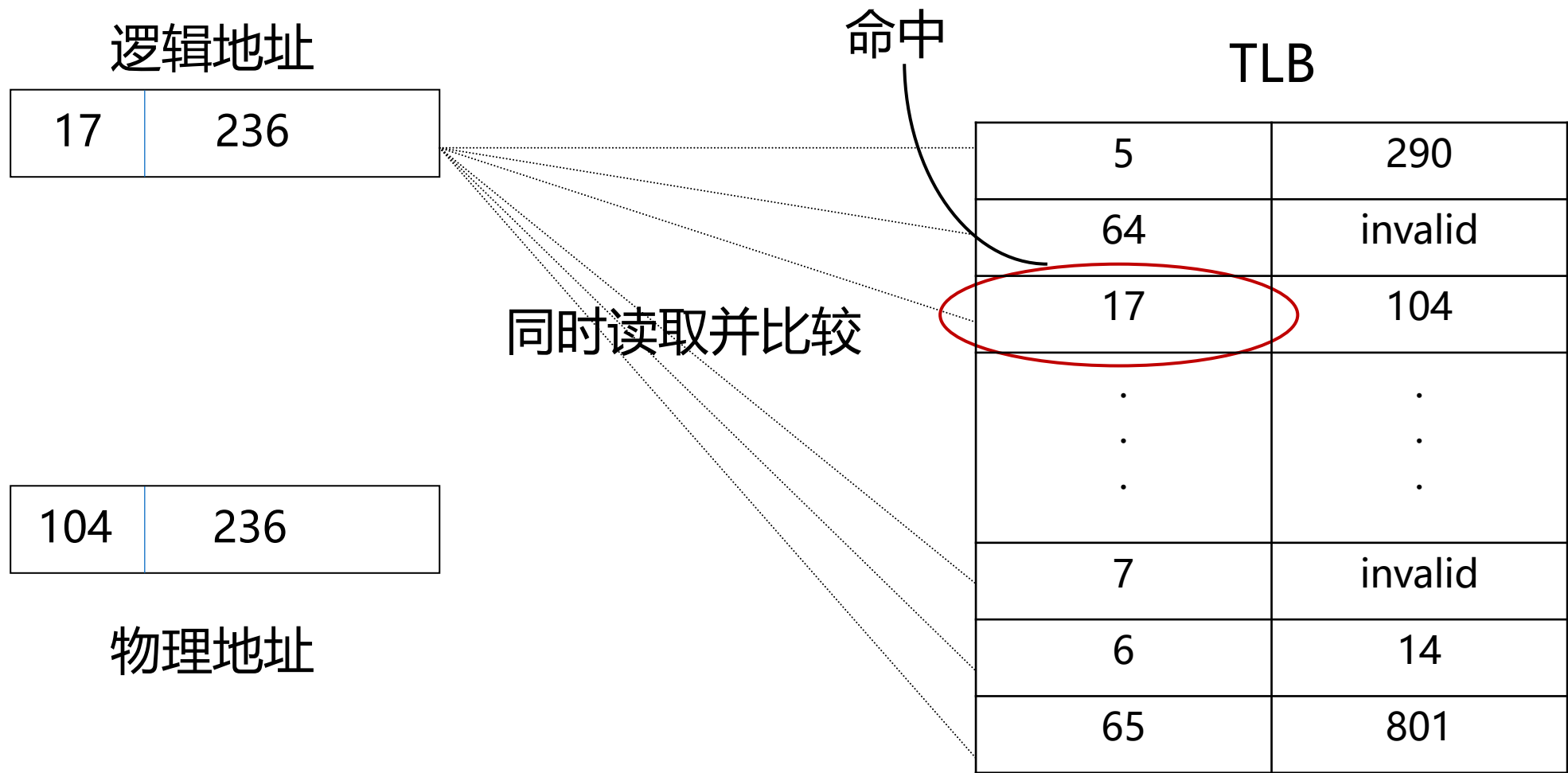


假设页框大小为4KB

具有快表的地址变换机构

- 采用分页存储管理，CPU访问内存，需访问两次：
 1. 访问页表
 2. 访问内存中的内容
- 为提高速度，增设一个具有并行查询能力的特殊Cache，称为TLB(Translation Lookaside Buffer)，用以存放当前访问的那些页表项
- TLB通常称为“快表”

具有快表的地址变换机构



具有快表的地址变换机构

■ 具有快表的地址转换

- 通过根据逻辑地址中的页号，查找快表中是否存在对应的页表项
- 若快表中存在该表项，称为命中，取出其中的页框号，加上页内偏移量，计算出物理地址
- 若快表中不存在该页表项，称为缺失，则再查找页表，找到逻辑地址中指定页号对应的页框号，同时，更新快表，将该表项插入快表中，并计算物理地址

具有快表的地址变换机构

- 例：有一分页系统，对主存的一次访问需要 $1.5\mu\text{s}$ ，实现一次页面访问的时间是多少？如果系统增加有快表，平均命中率为85%，当页表项在快表中时，其查找时间忽略为0，此时的访问时间是多少？
- 没有快表的访问时间 $=1.5 \times 2 = 3(\mu\text{s})$
- 增加快表后的访问时间 $=0.85 \times 1.5 + (1-0.85) \times 2 \times 1.5 = 1.725(\mu\text{s})$

两级和多级页表

- 对于大型程序页表可能很大，因为页表需要占用连续的内存空间，因而可能出现难以找到一块连续的大内存空间存放页表的问题
- 解决方法：页表本身离散化
- 将页表也分页，并对页表所占的空间进行索引形成外层页表(一般称为**页目录表**)，由此构成二级页表
- 更进一步可形成多级页表

两级和多级页表

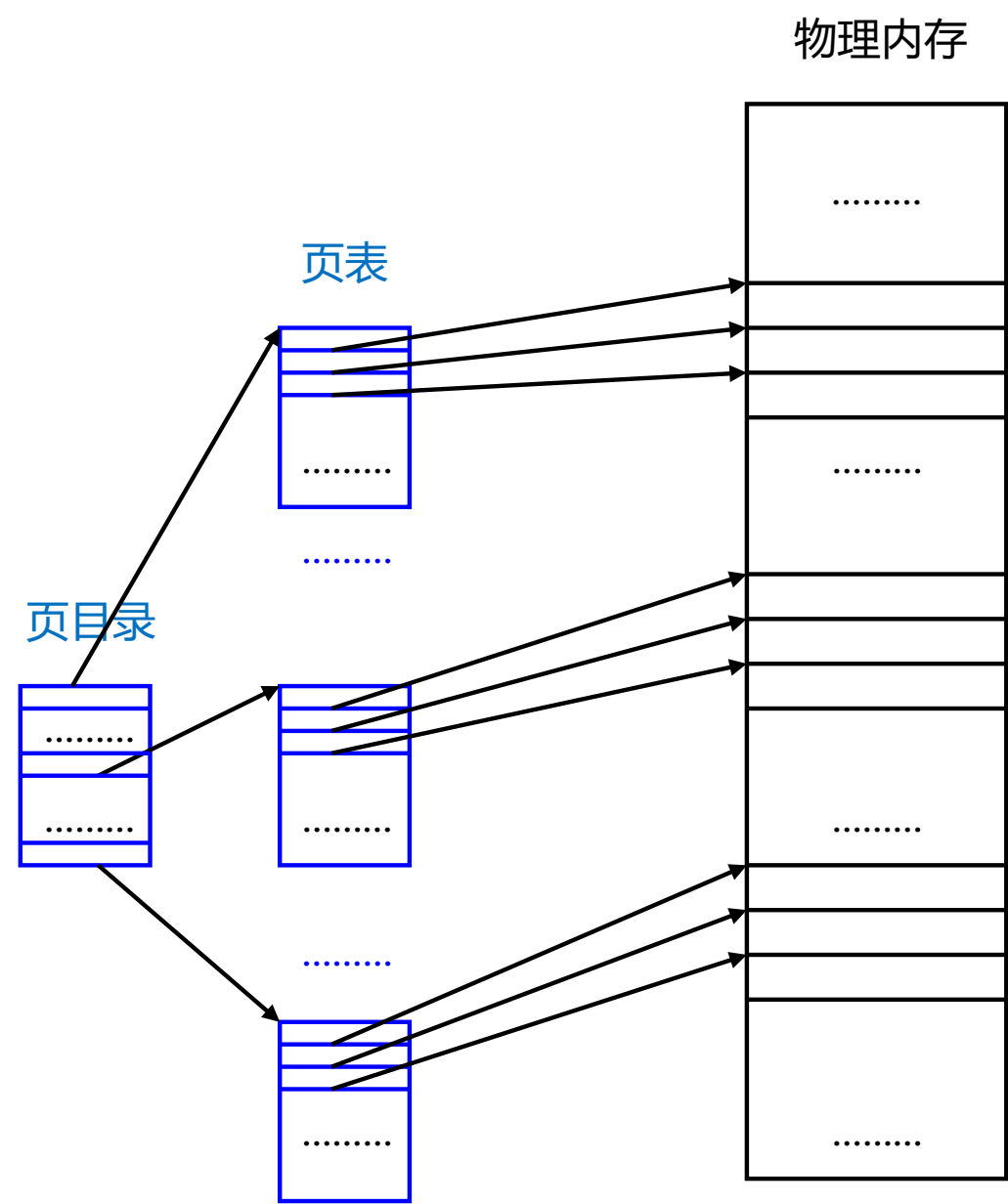
- 采用二级页表，则逻辑地址分为三部分：页目录索引，页表索引，页内偏移地址



逻辑地址格式

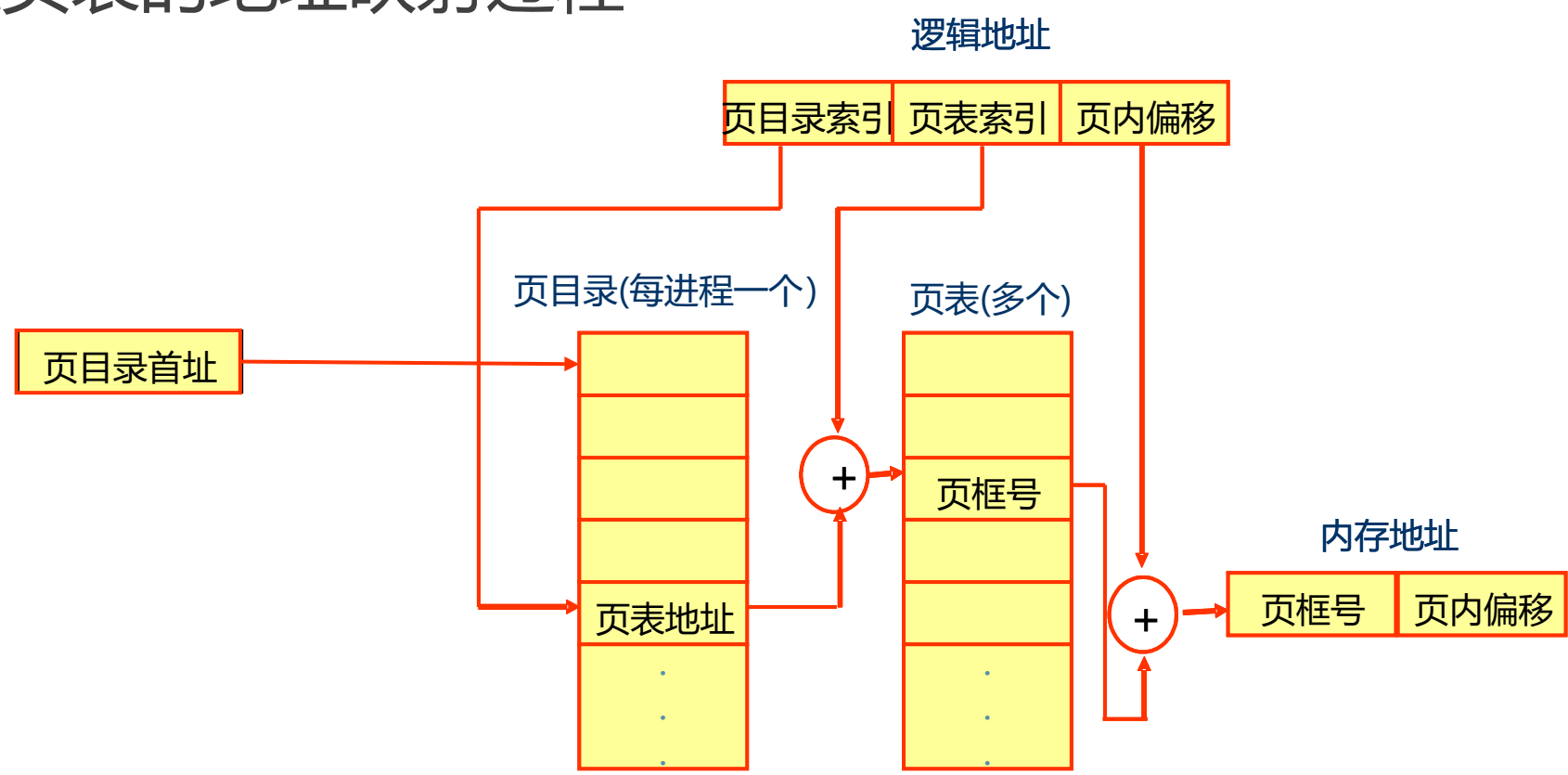
两级和多级页表

■二级页表



两级和多级页表

■二级页表的地址映射过程



页面与页框大小

- 分页系统中，页框的大小由计算机的硬件逻辑定义，通常页框的大小是2的幂次个字节，且常在512B~4kB之间，典型的页框大小为4kB
- 将页框大小设置为2的幂次，可以简化处理机中地址变换硬件逻辑的设计与实现
- 页面的大小由页框的大小决定

页面与页框大小

- 影响页面与页框大小的主要因素：内碎片、地址转换速度和页面交换效率
- 较小的页面有利于减少，从而有利于提高内存的利用率，然而，较小的页面也将导致页表过大，从而降低处理机访问页表时的命中率
- 页面越大，内外存之间的数据交换效率越高，因此，较大的页面有利于提高页面换入、换出内存的效率

分页式存储管理中的共享问题

- 分页式存储管理中不能实现真正意义共享
- 被共享的程序文本或数据不一定正好在一个或几个完整的页面中，有可能一个页面中既有允许共享的内容，又有不能共享的私有数据。因此，在分页环境下实现页面的共享比较困难

分页式存储管理的特点

- 存储分配的单位是页框
- 用户作业的逻辑地址空间按照页框的大小划分成页。这种划分是在系统内部进行的，用户感觉不到，即对用户是透明的
- 逻辑地址空间中的页可以进入内存中的任何一个空闲页框，并且分页式存储管理实行的是动态重定位，因此它打破了一个作业必须占据连续存储空间的限制，作业在不连续的存储区里，也能够得到正确的运行

简单页式存储管理

■优点：

- 没有外碎片，每个内碎片不超过页大小
- 一个程序不必连续存放
- 便于改变程序占用空间的大小——即随着程序运行而动态生成的数据增多，地址空间可相应增长

■缺点：

- 作业虽然不占据连续的存储区，但每次仍要求一次全部进入内存。因此，如果作业很大，其存储需求大于内存，那么还是存在小内存不能运行大作业的问题
- 存在内碎片
- 难于实现共享

某计算机采用页式存储管理，页的大小为1KB，物理内存大小为32页，某进程有8个逻辑页，那么逻辑地址有____位，物理地址有____位

- ☒ A 13, 15
- ☐ B 3, 5
- ☐ C 13, 13
- ☐ D 15, 15

页面大小为4KB，某进程有4个逻辑页，页表内容如图所示，则逻辑地址5318转换成物理地址为____

- ☒ A 5318
- ☐ B 1222
- ☐ C 9414
- ☐ D 越界

页号	页框号
0	2
1	1
2	3
3	7

提交

下述关于页式内存管理技术中页面大小的叙述，正确的是

- ☐ A 依据内存大小而定
- ☒ B 必须相同
- ☒ C 依据CPU的地址结构而定
- ☐ D 依据内存和外存大小而定

提交

大纲

01

非连续分配存储管理

02

简单页式存储管理

03

虚拟页式存储管理

虚拟存储器的引入

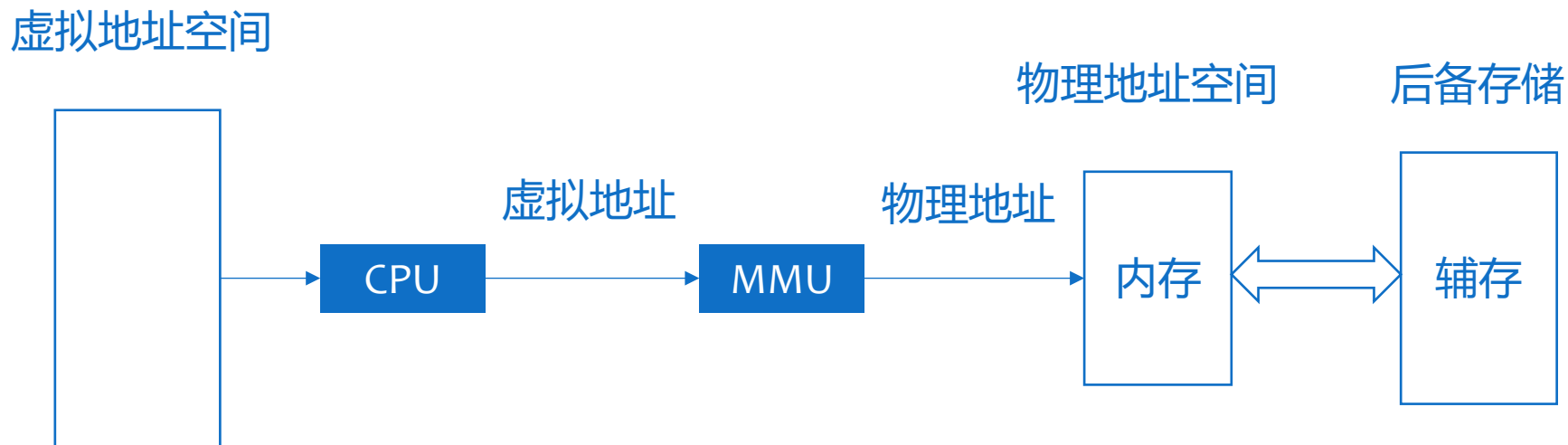
- 针对实存的管理方法，无论是连续的分区管理还是非连续的分页和分段管理都是将程序一次性全部装入内存后才能运行
- 因而可能会出现两种情况：
 - 一个特别大的进程，不能够一次装入内存
 - 系统中进程数太多，而又没有足够大的内存容纳这些进程，只能将少量进程装入内存运行，大量的进程在外存上等待

虚拟存储器的引入

- 解决基于实存的简单页式存储管理存在的问题的方法：
 - 从物理上扩大内存
 - 从逻辑上扩充内存——虚拟存储器
- 虚拟存储器的定义：在具有层次结构存储体系的计算机系统中，采用自动实现部分装入和部分交换功能，为用户提供一个独立于物理内存容量的、可寻址的“存储器”

虚拟存储器的引入

- 进程虚拟地址空间的大小由处理机的地址结构所决定，与主存大小并无直接关系
- 虚拟存储器以辅助存储器作为后备存储器



虚拟存储器的引入

■工作原理：

- 以大容量的辅助存储器做后备存储器
- 把进程保存在后备存储器上，当要求装入时，只将其中一部分先装入主存，另一部分暂时存放在后备存储器上，进程执行过程中要用到那些不在主存中的信息时，再把它们装到主存中
- 问题：在进程不全部装入主存的情况下能否保证正确执行——由存储器访问的局部性原理做保障

引入虚拟存储技术的好处

- **大程序**：可在较小的可用内存中执行较大的用户程序
- **大的用户地址空间**：提供给用户可用的虚拟内存空间可以大于物理内存或实存(real memory)
- **并发**：可在内存中容纳更多程序并发执行

虚拟存储器的引入

■ 实现虚拟存储器必须解决的问题：

- 怎样知道当前哪些信息已在主存储器中，哪些信息尚未装入主存中？
- 如果进程要访问的信息不在主存储器中，怎样去找到这些信息且把它们装入到主存中？
- 在把欲访问的信息装入主存时，发现主存储器中已无空闲区域，又该怎么办？

虚拟页式存储管理

- 由于存储器访问的局部性，在程序装入时，不必将其全部读入到内存，而只需将当前需要执行的部分页读入到内存，就可让程序开始执行
- 在程序执行过程中，如果需执行的指令或访问的数据尚未在内存(称为缺页中断)，则由处理器通知操作系统执行缺页中断处理程序，将相应的页调入到内存，然后继续执行程序
- 另一方面，操作系统可以将内存中暂时不使用的页调出保存在外存上，从而腾出空间存放将要装入的程序以及将要调入的页面

虚拟页式存储管理

- 虚拟页式存储管理是在在简单页式存储管理的基础上，增加调页和页面置换功能
- 对进程页表的修改——需要在进程页表中添加若干项
 - 标志位：存在位(present bit)、修改位(modified bit)
 - 访问统计：在近期内被访问的次数，或最近一次访问到现在的时间间隔
 - 外存地址

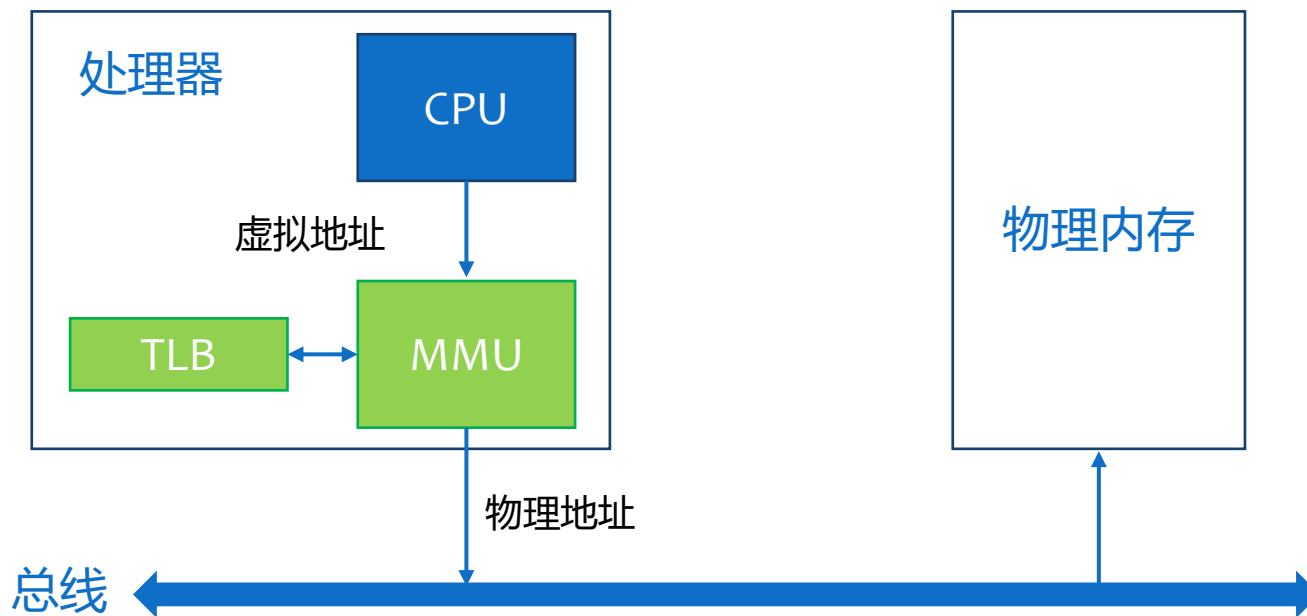
Page Table Entry

P	M	Other Control Bits	Frame Number
----------	----------	---------------------------	---------------------

虚拟页式存储管理的硬件支持

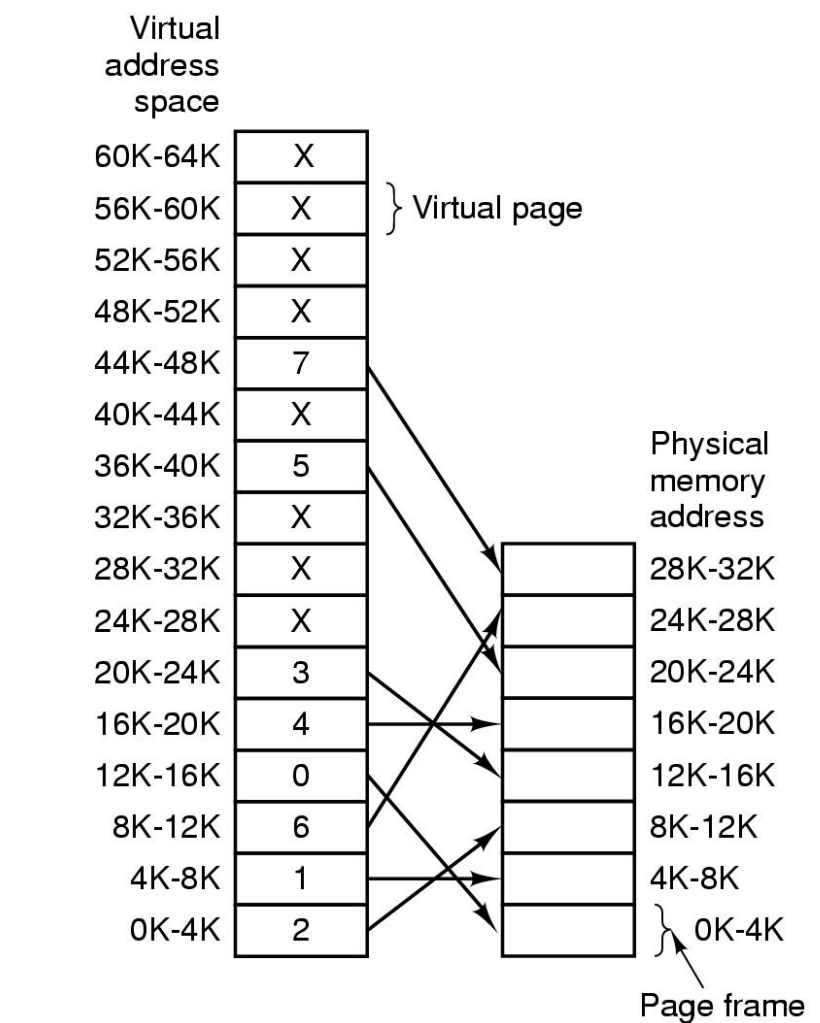
■ MMU(Memory Management Unit, 内存管理单元)

- 管理硬件页表寄存器
- 分解虚拟地址
- 访问页表
- 管理快表TLB
- 设置和检查页表中各个特征位
- 发出缺页中断，并将控制权交给内核存储管理处理



虚拟页式存储管理的硬件支持

虚拟地址与物理地址的映射关系



假设页框大小为4KB，虚拟地址空间大小64KB，物理地址空间大小32KB

虚拟页式存储管理的硬件支持

- 缺页中断是一种特殊的中断
- 主要表现在：
 - 在指令执行期间产生和处理中断信号。而普通中断是在每条指令结束后去检查是否有中断到达
 - 一条指令执行期间，可能产生多次缺页中断。一条指令在执行时，指令本身和操作数可能都不在内存中

页面调入策略

- 页面调入策略考虑何时将页面装入主存，有两种策略：
- 请求调页(demand paging)：只调入发生缺页时所需的页面
 - 优点：容易实现，节省物理内存
 - 缺点：处理缺页中断和调页的系统开销较大，每次仅调一页，增加了磁盘I/O次数
- 预先调页(prepaging)：在发生缺页前预先调入页面
 - 优点：减少缺页中断
 - 缺点：基于预测，若调入的页在以后很少被访问，则效率低
 - 常用于程序装入时的调页

页面调入策略

- 从何处调入？
- 虚拟页式存储管理系统中，把后备存储器分为两部分：文件区和对换区
- 实现方式：
 - 进程的所有页面都放在对换区
 - 将修改过的页面放在对换区，未改的放在文件区

页面分配策略

■ 页面分配指系统为进程分配主存页框，需考虑的因素：

1. 分给进程的页框越少，同一时间处于内存的进程就越多，至少有一个进程处于就绪态的可能性就越大
2. 如果进程只有小部分在主存里，即使局部性很好，缺页中断率还会相当高
3. 因程序的局部性原理，分给进程的内存超过一定限度后，再增加内存空间，不会明显降低进程的缺页中断率

页面分配策略

- 页面分配策略包括固定分配和可变分配
- **固定分配**：进程保持页框数固定不变
 - 进程创建时，根据进程类型和程序员的要求决定页框数，只要有一个缺页中断产生，进程就会有一页被替换
- **可变分配**：进程分得的页框数可变
 - 进程执行的某阶段缺页率较高，说明目前局部性较差，系统可多分些页框以降低缺页率，反之说明进程目前的局部性较好，可减少分给进程的页框数

页面置换策略

- 如果欲调入一页时，主存中已没有空闲页框，怎么办？
- 必须先调出已在主存中的某一页，再将当前所需的页调入，同时对页表作相应的修改
- 采用某种算法选择一页暂时调出，把它存放到后备存储器上去，让出主存空间，用来存放当前要使用的页面，这一过程称为[页面置换](#)或[页面调度](#)
- 若被页面调度选中调出的页又要被访问，则可用类似的方法调出另一些页面而将其调入

页面置换策略

- 页面置换算法的选择非常重要。如果选用了不合适页面置换算法就会出现这样的现象：刚被调出的页又立即要用，因而又要把它调入；而调入不久又被调出；调出不久又再次被调入
- 如此反复，使调度非常频繁，以至于使大部分时间都花费在来回调度上，这种现象称为**颠簸或抖动(thrashing)**
- 因而应该选择一种好的页面置换算法，以减少和避免抖动现象

页面置换策略

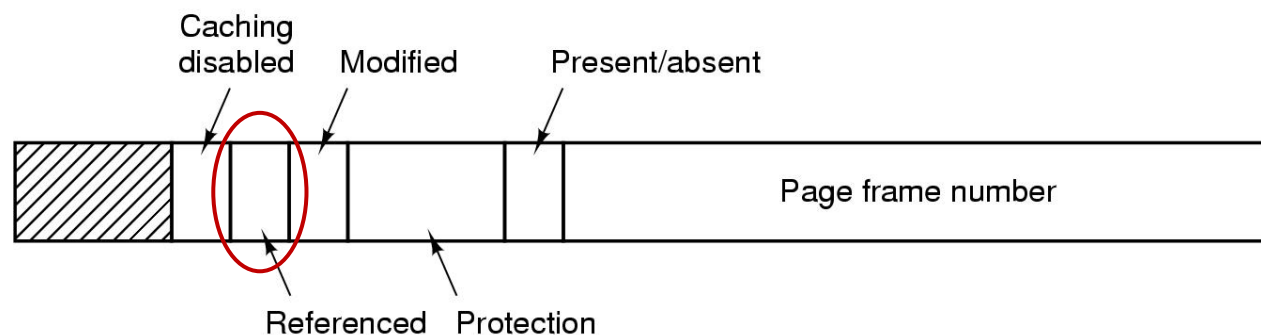
- 页面置换算法的目标：把未来不再使用的或短期内不使用的页面调出
- 通常只能在局部性原理指导下依据过去的统计数据进行分析预测
- 常用算法
 - 最优页面置换算法OPT
 - 最近未使用页面置换算法NRU
 - 先进先出页面置换算法FIFO
 - 第二次机会页面置换算法
 - 时钟页面置换算法Clock
 - 最近最少使用页面置换算法LRU
 - 最不常用页面置换算法NFU
 - 老化(aging)算法

最优页面置换算法OPT

- 选择未来不再使用的或在离当前最远时间才使用的页面被置换
- 这是一种理想情况，是实际执行中无法预知的，因而不能实现。可用作性能评价的依据

最近未使用页面置换算法NRU

■在页表项中设置页面使用情况的状态位



■R位被定期地清零，以区别最近没有被访问的页面和被访问的页面

最近未使用页面置换算法NRU

■根据R位和M位，可以把页面分成四类

0) not referenced, not modified

1) not referenced, modified

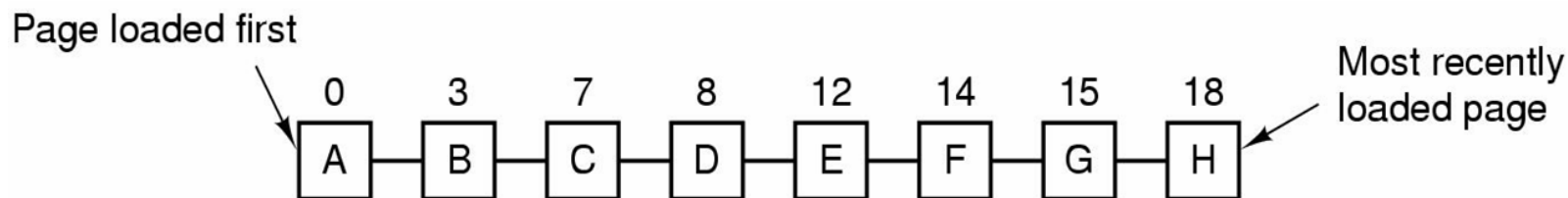
2) referenced, not modified

3) referenced, modified

■NRU(Not Recently Used, 最近未使用)算法随机地从编号最小的非空类中挑选一个页面淘汰之

先进先出页面置换算法FIFO

- 先进先出(FIFO)算法选择最老的页面被置换
- 可以通过链表来表示各页的装入时间先后



先进先出页面置换算法FIFO

■性能较差

- 较早调入的页往往是经常被访问的页，这些页在FIFO算法下可能被反复调入和调出
- 存在Belady异常：如果对一个进程未分配它所要求的全部页，有时会出现分配的页框数增多，缺页率反而提高的异常现象

先进先出页面置换算法FIFO

■ Belady异常的例子

- 进程P有5个虚拟页面，编号从0到4，页面访问的次序如下：0 1 2 3 0 1 4 0 1 2 3 4
- 如果在内存中分配3个页框，则缺页情况如下：12次访问中有缺页9次

All pages frames initially empty

	0	1	2	3	0	1	4	0	1	2	3	4	
Youngest page		0	1	2	3	0	1	4	4	4	2	3	3
			0	1	2	3	0	1	1	1	4	2	2
Oldest page				0	1	2	3	0	0	0	1	4	4
		P	P	P	P	P	P				P	P	

9 Page faults

先进先出页面置换算法FIFO

■ Belady异常的例子

- 进程P有5个虚拟页面，编号从0到4，页面访问的次序如下：0 1 2 3 0 1 4 0 1 2 3 4
- 如果在内存中分配4个页框，则缺页情况如下：12次访问中有缺页10次

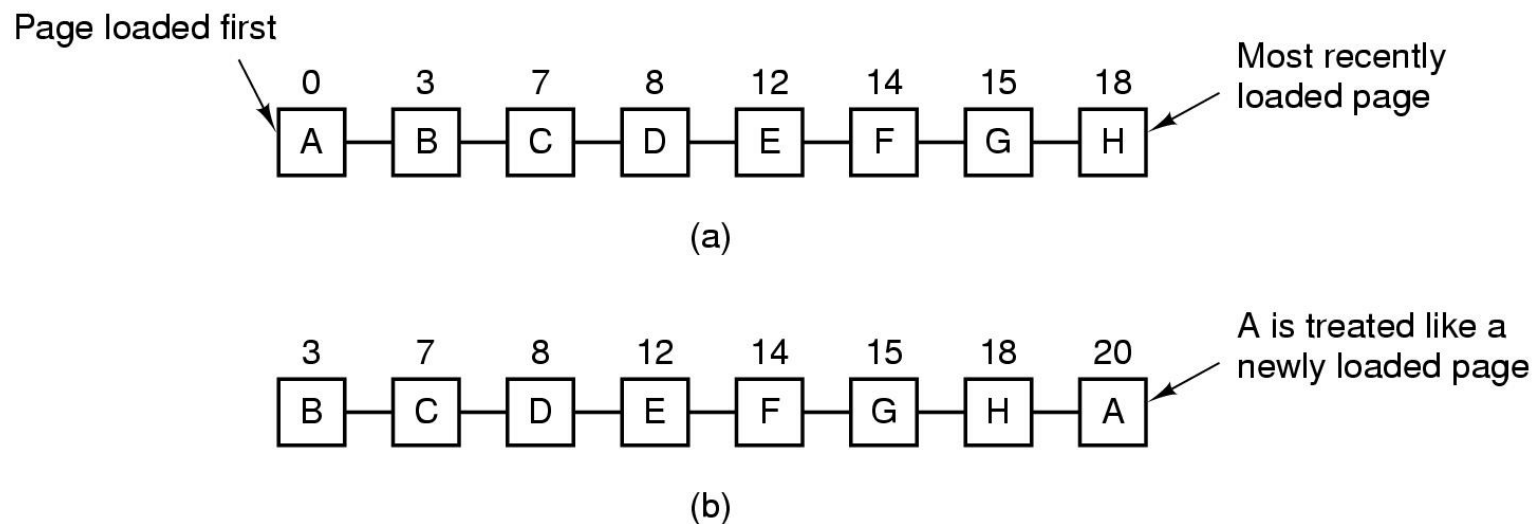
		0	1	2	3	0	1	4	0	1	2	3	4	
Youngest page		0	1	2	3	3	3	4	0	1	2	3	4	
			0	1	2	2	2	3	4	0	1	2	3	
Oldest page				0	1	1	1	2	3	4	0	1	2	
					0	0	0	1	2	3	4	0	1	
		P	P	P	P			P	P	P	P	P	P	

10 Page faults

第二次机会页面置换算法

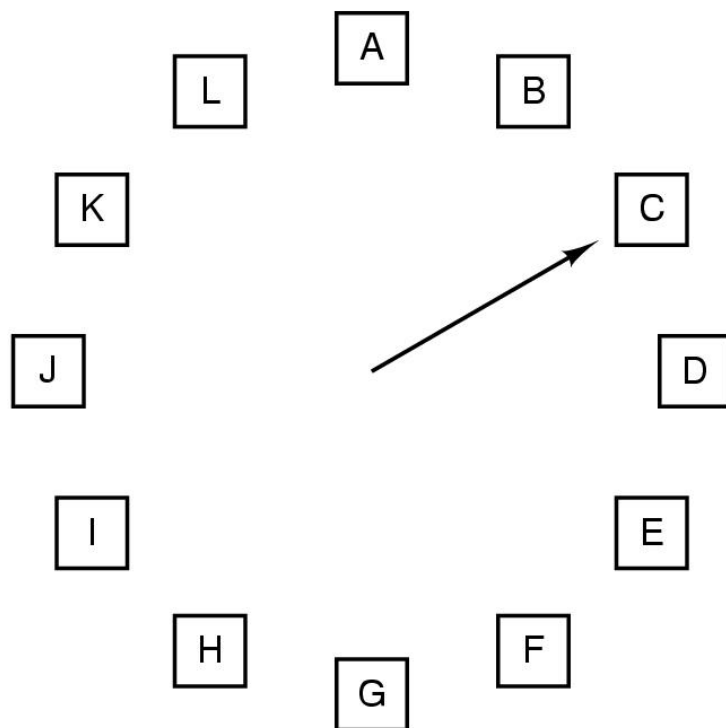
■对FIFO算法的改进：

- 检查最老页面的R位，如果为0，则置换掉；
- 如果为1，则清除该位，并将该页放到链表的末尾



时钟页面置换算法Clock

■逻辑上与第二次机会页面置换算法等价



When a page fault occurs, the page the hand is pointing to is inspected. The action taken depends on the R bit:

R = 0: Evict the page

R = 1: Clear R and advance hand

最近最少使用页面置换算法LRU

- 最近最少使用(Least Recently Used, LRU)
- 选择内存中最久未使用的页面被置换。这是局部性原理的合理近似，性能接近最佳算法
- 由于需要记录页面使用时间的先后关系，硬件开销教大

最近最少使用页面置换算法LRU

■ LRU算法硬件机构(1)

- 硬件有一个计数器C(C必须足够长, 例如64位), 每条指令执行后自动增1
- 每个页表项有一个能够容纳计数器值的域
- 每次访问内存后, 当前C值被复制到被访问页面的页表项中
- 页表项的计数值最小的页面就是最近最少使用的页面

最近最少使用页面置换算法LRU

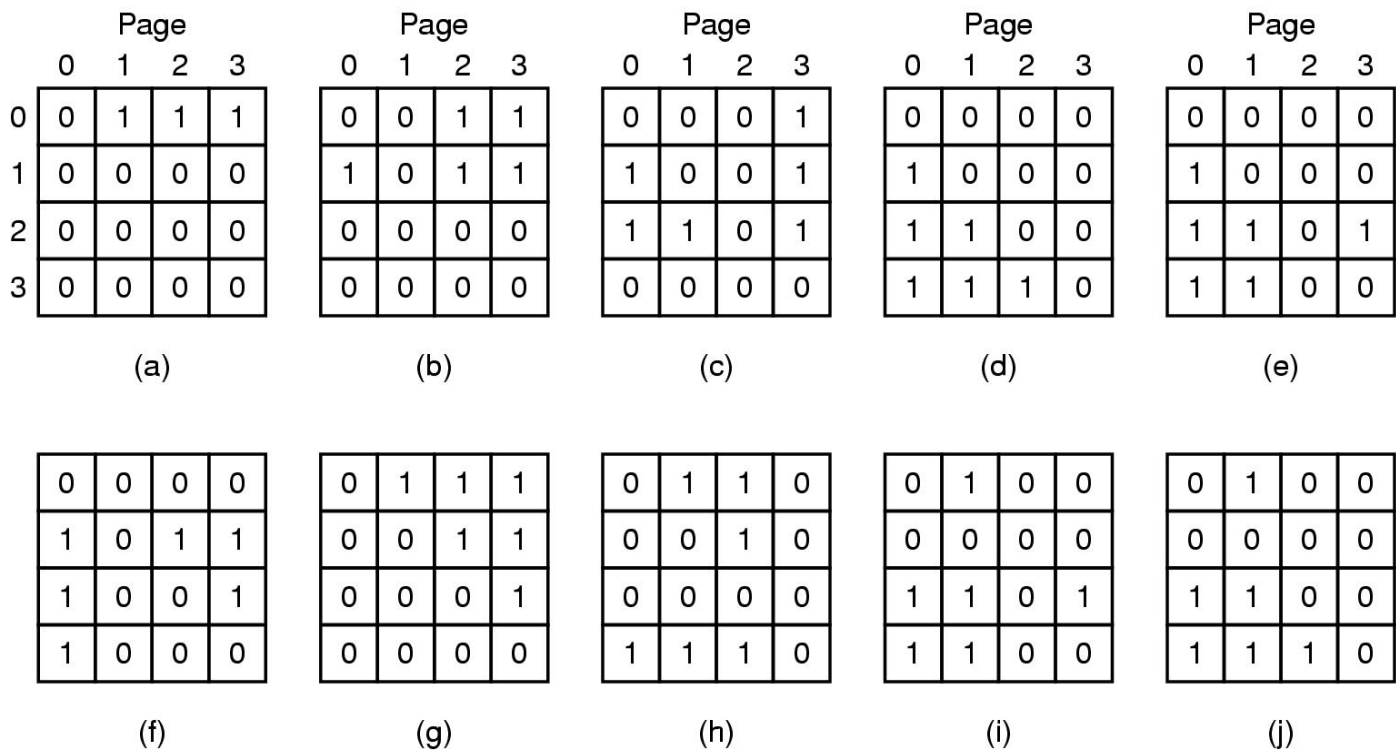
■ LRU算法硬件机构(2)

- 对于有 n 个页框的机器，LRU硬件可以维护一个 $n \times n$ 矩阵，矩阵元素初值为0
- 访问页框 k 时，先把 k 行的位都置1，再把 k 列的位都清0
- 任何时刻，二进制值最小的行就是最近最少使用的

最近最少使用页面置换算法LRU

LRU算法硬件机构(2)

例4个页框，访问顺序0 1 2 3 2 1 0 3 2 3



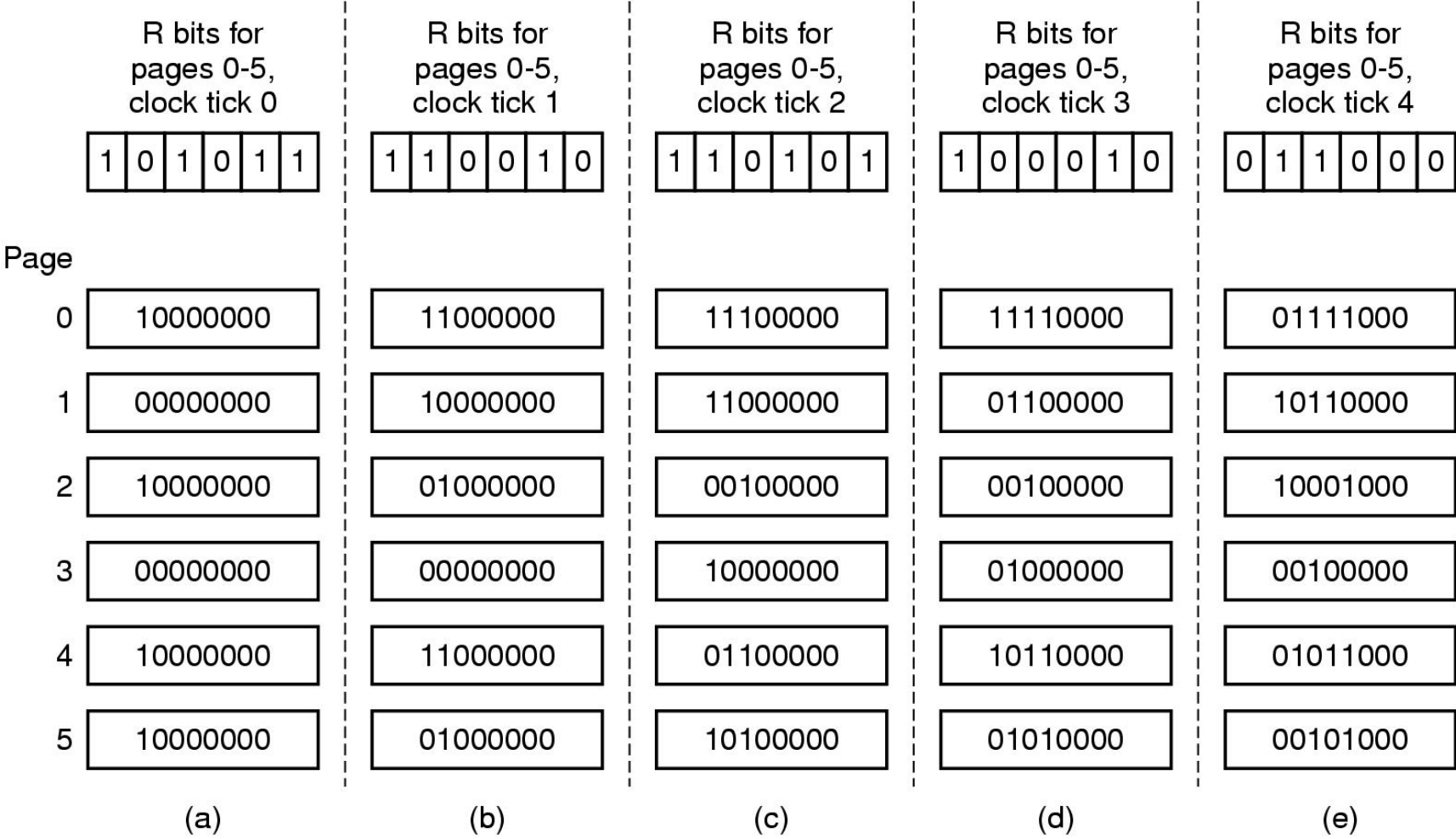
最不常用页面置换算法NFU

- 用软件模拟LRU——NFU(Not Frequently Used, 最不常用)算法
- 将每个页面与一个软件计数器相关联
- 每次时钟中断时, 将页表项的R位加到计数器上
- 发生页面失效时, 计数器值最小的页面被置换

老化(aging)算法

- 对NFU算法的改进
- 在R位被加进之前先将计数器右移一位，然后将R位加到计数器的最左端
- 发生页面失效时，计数器值最小的页面被置换

老化(aging)算法



页面置换算法举例

■某程序在内存中分配三个页框，初始为空，页面访问顺序为4 3 2 1 4 3 5 4 3 2 1 5 5 4 3 2 1 5

共缺页中断7次	OPT	4	3	2	1	4	3	5	4	3	2	1	5
		4	4	4	4	4	4	4	4	4	4	4	4
			3	3	3	3	3	3	3	3	2	1	1
				2	1	1	1	5	5	5	5	5	5
		x	x	x	x	√	√	x	√	√	x	x	√

页面置换算法举例

■某程序在内存中分配三个页框，初始为空，页面访问顺序为4 3 2 1 4 3 5 4 3 2 1 5 5 4 3 2 1 5

共缺页中断10次

LRU	4	3	2	1	4	3	5	4	3	2	1	5
	4	4	4	1	1	1	5	5	5	2	2	2
		3	3	3	4	4	4	4	4	4	1	1
			2	2	2	3	3	3	3	3	3	5
	x	x	x	x	x	x	x	√	√	x	x	x

页面置换算法举例

■某程序在内存中分配三个页框，初始为空，页面访问顺序为4 3 2 1 4 3 5 4 3 2 1 5 5 4 3 2 1 5

共缺页中断9次

FIFO	4	3	2	1	4	3	5	4	3	2	1	5
	4	4	4	1	1	1	5	5	5	5	5	5
		3	3	3	4	4	4	4	4	2	2	2
			2	2	2	3	3	3	3	3	1	1
	x	x	x	x	x	x	x	√	√	x	x	√

页面置换算法举例

■某程序在内存中分配三个页框，初始为空，页面访问顺序为4 3 2 1 4 3 5 4 3 2 1 5 5 4 3 2 1 5

FIFO	4	3	2	1	4	3	5	4	3	2	1	5
共缺页中断9次	4	4	4	1	1	1	5	5	5	5	5	5
		3	3	3	4	4	4	4	4	2	2	2
			2	2	2	3	3	3	3	3	1	1
	x	x	x	x	x	x	x	√	√	x	x	√

工作集

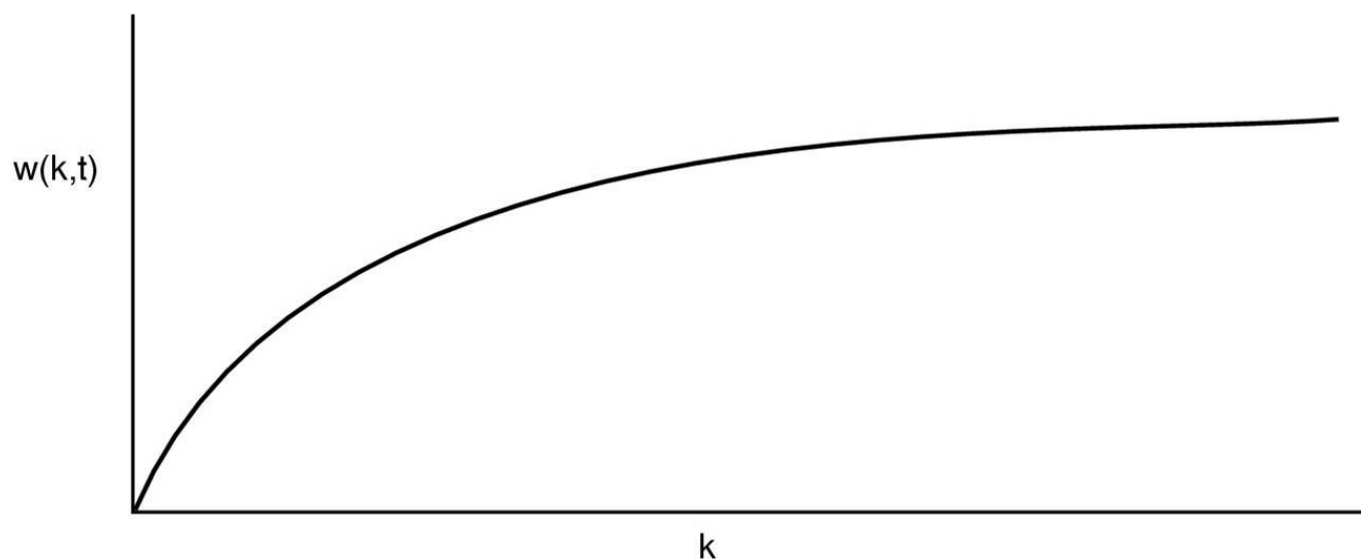
- 上述页面置换算法都属**固定分配方式下的局部置换算法**——为进程分配的页框数是固定的，当缺页中断产生时，只在本进程分配的页中进行置换
- 产生抖动的原因：
 - 页面置换算法不合理
 - 分配给进程的物理页框数太少
- 为此，P·J·Denning于1968年提出了工作集模型

工作集

- 根据存储器访问的局部性原理，一般情况下，进程在一段时间内总是集中访问一些页面，这些页面称为活跃页面，如果分配给一个进程的物理页面数太少，使该进程所需的活跃页面不能全部装入内存，则进程在运行过程中将频繁发生中断
- 如果能为进程提供与活跃页面数相等的物理页面数，则可减少缺页中断次数
- 例： 26157775162341234443434441327
 |←————→|t1 |←————→|t2

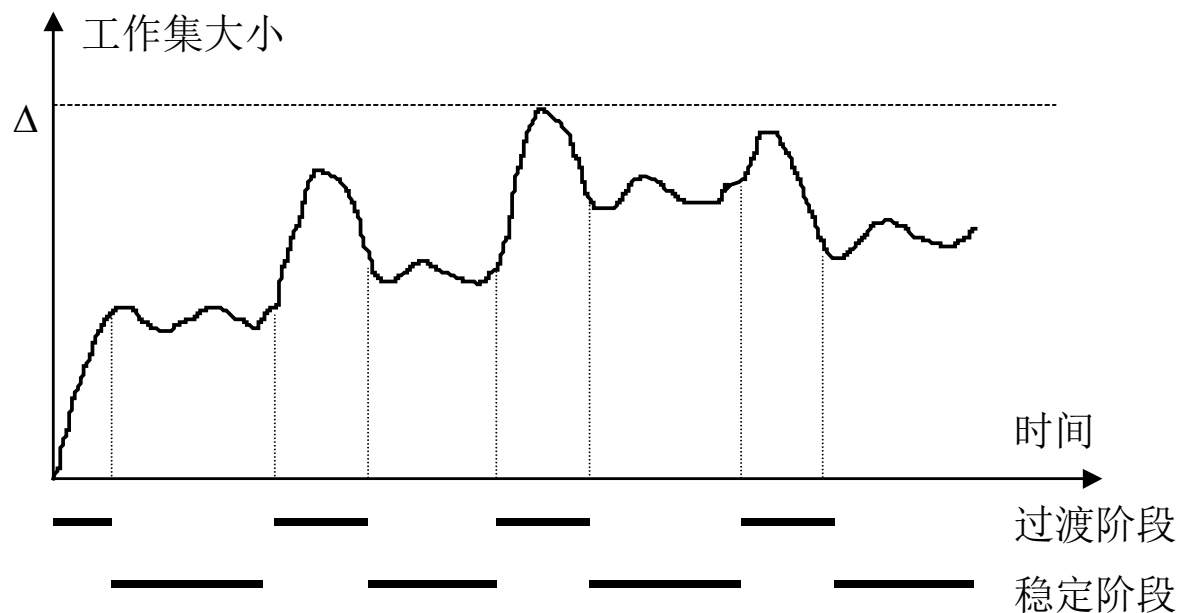
工作集

- 工作集(working set)：一个进程当前正在使用的页面的集合
- 工作集可用一个二元函数 $W(k, t)$ 表示——在时刻 t ，所有最近 k 次内存访问所用过的页面



工作集

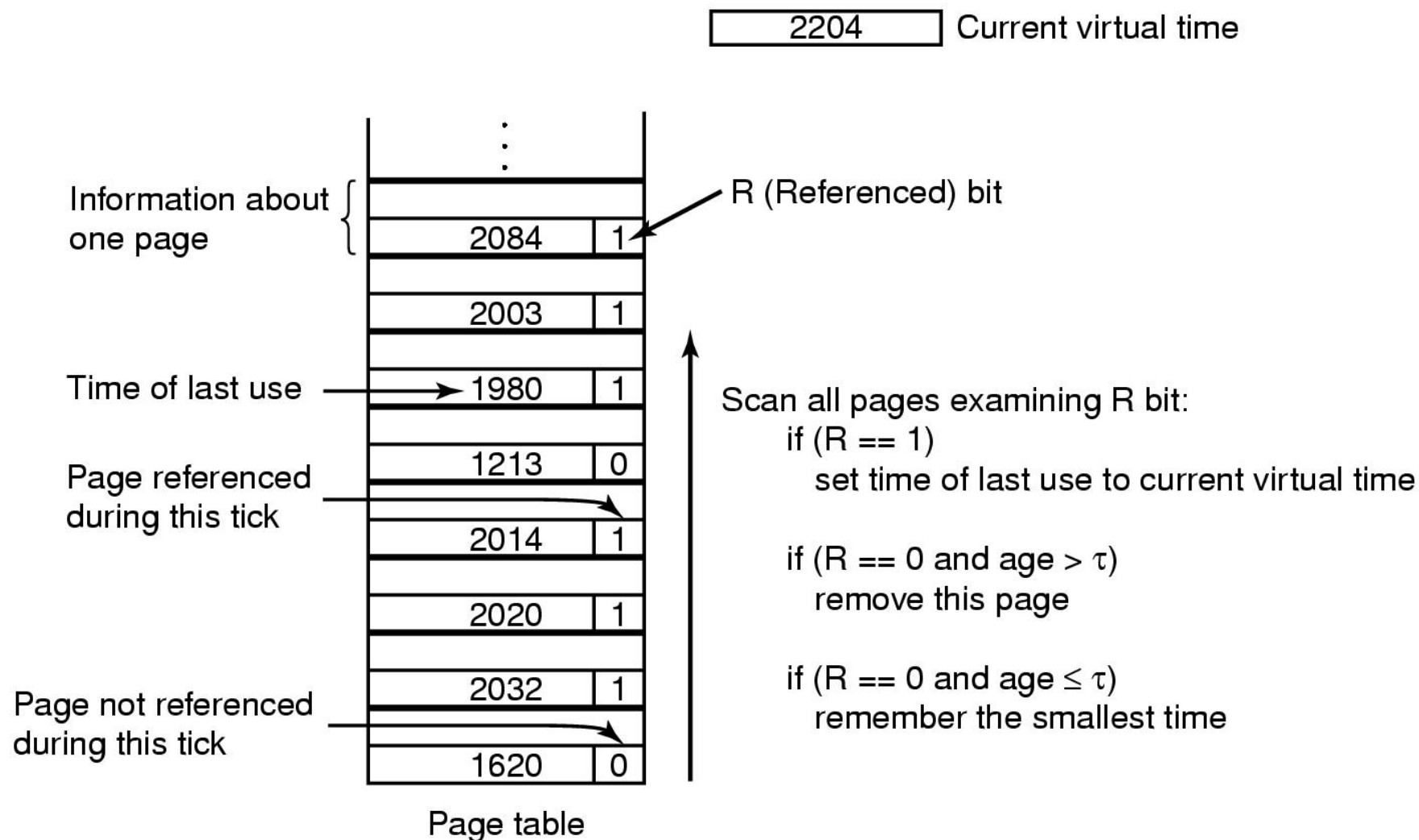
- 工作集大小的变化：进程开始执行后，随着访问新页面逐步建立较稳定的工作集。当内存访问的局部性区域的位置大致稳定时，工作集大小也大致稳定；局部性区域的位置改变时，工作集快速扩张和收缩过渡到下一个稳定值



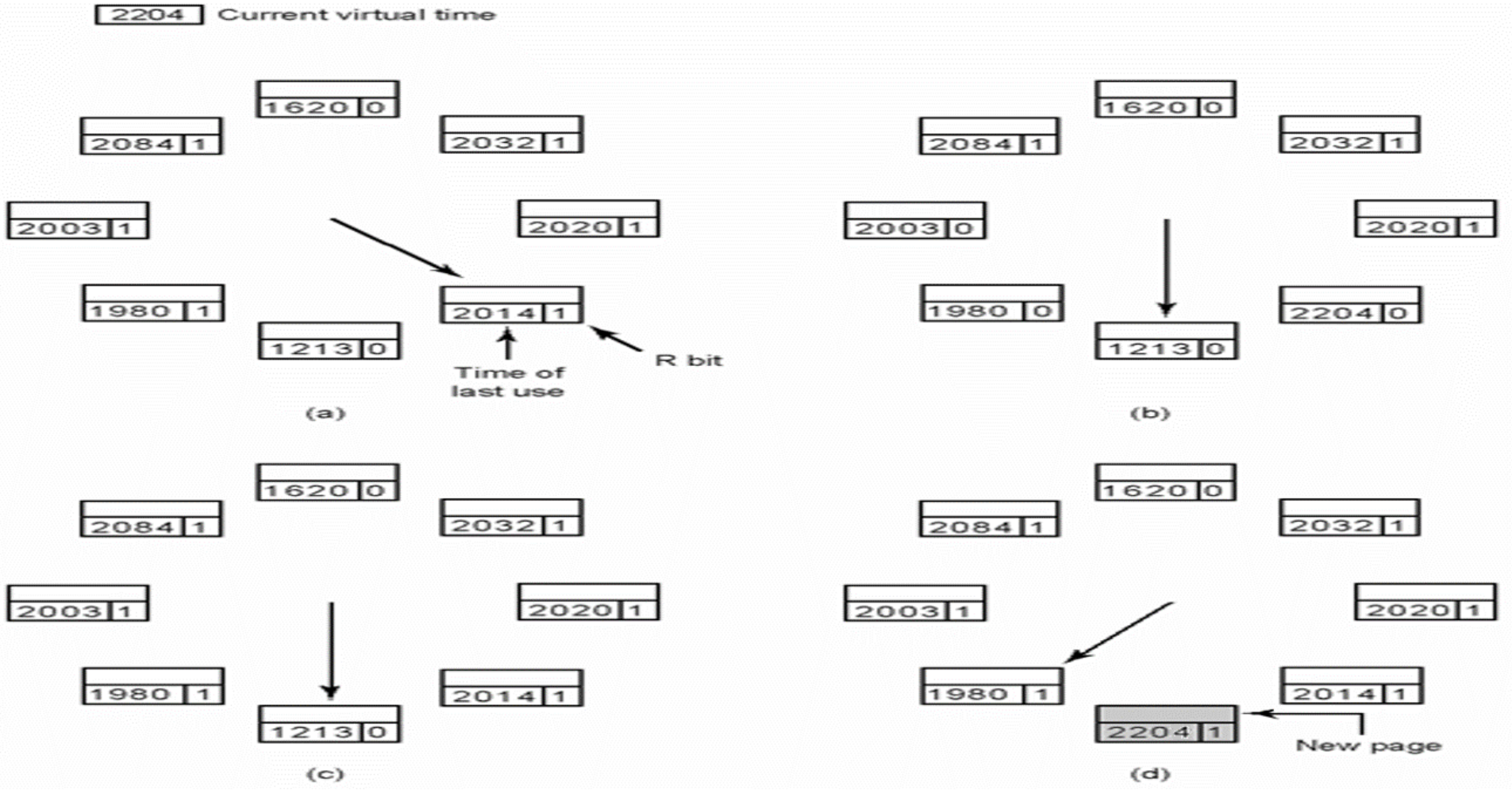
工作集

- 工作集的一种近似——用二元函数 $W(t, \tau)$ 表示工作集
 - t 是当前时刻
 - τ 是一个虚拟时间段，称为窗口大小(window size)，它采用“虚拟时间”单位
 - 工作集是在 $[t - \tau, t]$ 时间段内所访问的页面的集合
- 一个进程从它开始执行到当前时刻所实际使用的CPU时间总数称为当前虚拟时间

工作集页面置换算法



工作集钟表页面置换算法



工作集

- 采用工作集策略的困难在于操作系统要经常监视活动进程的行为和进程缺页中断率的情况，会增加系统的开销

页面置换算法小结

算法	评价
OPT	不可实现，用于基准测试
NRU	非常简陋
FIFO	可能替换掉重要的页
二次机会	对FIFO的重大改进
Clock	等价于二次机会
LRU	性能优异，但是难于实现
NFU	LRU的近似
老化	有效近似LRU的算法
工作集	实现代价较大
WSClock	良好的性能

页面清除策略

- 清除策略考虑何时把一个修改过的页面写回辅助存储器
- 请求清除(demand cleaning)是仅当一页选中被替换，且之前它又被修改过，才把这个页面写回辅存
 - 优点：容易实现
 - 缺点：写出一页是在读进新页前进行的，要成双操作，引起进程等待两次I/O操作，会降低CPU使用效率
- 预清除(precleaning)：页写出外存，但是仍在内存中，直到页替换算法选中它。预清除可以成批地把页面写出
 - 优点：减少I/O次数
 - 缺点：如若刚写出了很多页面，在被替换前，其中大部分又被更改，预清除就毫无意义

页面清除策略

- **页缓冲(page buffering)**: 使清除操作和替换操作不必成双进行。在页缓冲中, 替换了的页面进入两个队列: 修改页面队列和非修改页面队列
- 修改页面队列中的页的周期性地成批写出并加入到非修改页面队列
- 非修改页面队列中的页面, 当它被再次引用时回收, 或者分配给其他页

简单页式存储管理 vs 虚拟页式存储管理

特性	简单页式内存管理	虚拟页式内存管理
是否使用磁盘	否，仅使用物理内存	物理内存 + 磁盘
缺页中断	无	有
页面置换	无	有（LRU、FIFO 等）
内存分配限制	进程大小 ≤ 物理内存	可超过物理内存
适用场景	嵌入式系统、实时系统	现代通用操作系统

虚拟存储器的容量_____。

- ☐ A 为内存和外存容量之和
- ☒ B 由CPU的地址结构决定
- ☐ C 任意
- ☐ D 由进程的地址空间决定

提交

在分页虚拟存储技术中，二次机会和时钟算法在决定淘汰哪一页时，都用到了____

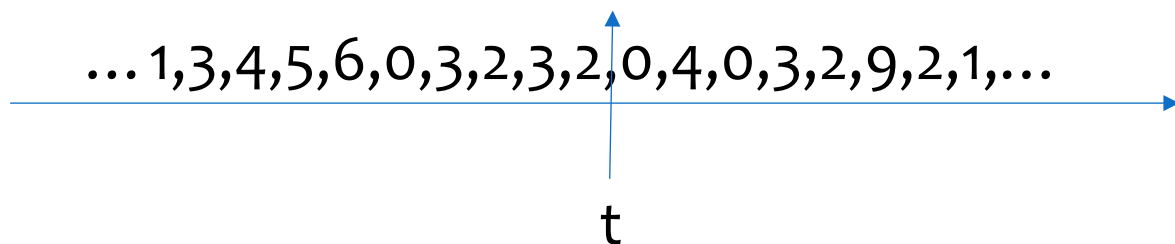
- ☒ A 引用位
- ☐ B 修改位
- ☐ C 快表
- ☐ D 地址变换

采用FIFO页面置换算法时，当分配的页框数增加时，缺页中断次数_____。

- ☐ A 减少
- ☐ B 增加
- ☒ C 可能减少也可能增加
- ☐ D 无关

某进程页面访问序列如下，若工作集窗口大小为6，则t时刻的工作集为_____。

- ☒ A {0, 2, 3, 6}
- ☐ B {0, 2, 3, 4}
- ☐ C {0, 2, 3, 4, 9}
- ☐ D {0, 2, 3, 4, 5, 6}



某页式虚拟内存系统经测试发现CPU利用率为10%，磁盘的利用率为95%，说明_____。

- ☐ A 系统工作正常
- ☐ B 需提高系统并发度
- ☐ C 需降低系统并发度
- ☒ D 系统可能出现内存抖动

操作系统，2026

谢谢

OPERATING SYSTEM